



Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης

Πολυτεχνική Σχολή

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Ενσωματωμένα Συστήματα Πραγματικού Χρόνου

Τελική Εργασία

Άγγελος-Χαράλαμπος Μπίλλης

cmpillis@ece.auth.gr

AEM: 10998

13 Ιουλίου 2026

1 Εισαγωγή

Στην παρούσα εργασία καλούμαστε να υλοποιήσουμε ένα πολυνηματικό σύστημα πραγματικού χρόνου σε C, το οποίο συνδέεται στο WebSocket firehose του Bluesky Jetstream, επεξεργάζεται ασύγχρονα τα εισερχόμενα JSON μηνύματα και καταγράφει μετρικές τηλεμετρίας κάθε ένα δευτερόλεπτο, με στόχο τη συνεχή λειτουργία για 24 ώρες σε Raspberry Pi Zero 2W. Θα ακολουθήσουμε την αρχιτεκτονική Producer–Consumer με τρία POSIX threads, θα σχεδιάσουμε τον συγχρονισμό τους ώστε να αποφευχθούν race conditions, και θα αξιολογήσουμε την απόδοση του συστήματος ως προς το jitter, το drift, τη χρήση CPU και την ανθεκτικότητά του σε διακοπές δικτύου. Για τη σύνδεση χρησιμοποιήθηκε η βιβλιοθήκη libwebsockets (μεταγλωττισμένη από τον πηγαίο κώδικα στο Pi) και για το parsing η cJSON.

2 Αρχιτεκτονική του Συστήματος

2.1 Τα τρία νήματα

Το πρώτο νήμα, ο **Παραγωγός**, τρέχει το event loop της libwebsockets. Κάθε φορά που φτάνει ένα νέο JSON frame από το δίκτυο, το τοποθετεί στον κυκλικό buffer και ξυπνάει τον Καταναλωτή μέσω condition variable, επιστρέφοντας αμέσως στο δίκτυο. Ένα πρακτικό ζήτημα που χρειάστηκε προσοχή είναι ότι τα μεγάλα μηνύματα ενδέχεται να παραδοθούν από τη βιβλιοθήκη σε κομμάτια (fragments)· τα συναρμολογούμε λοιπόν σε έναν ενδιάμεσο buffer και τα προωθούμε μόνο όταν η `lws_is_final_fragment()` επιστρέψει αληθές, ώστε ο Καταναλωτής να μη λάβει ποτέ μισό JSON. Αν ο buffer είναι γεμάτος, το μήνυμα απορρίπτεται και καταμετράται. Έτσι, ο Παραγωγός δεν μπλοκάρει ποτέ, ώστε να μη χάνονται πακέτα από το socket.

Το δεύτερο νήμα, ο **Καταναλωτής**, κοιμάται σε `pthread_cond_wait` μέχρι να υπάρξουν δεδομένα. Μόλις ξυπνήσει, αντιγράφει το μήνυμα σε τοπική μνήμη, ξεκλειδώνει το mutex και μετά εκτελεί το parsing, ώστε η ακριβή αυτή διαδικασία να γίνεται εκτός κρίσιμου τμήματος. Διαβάζει το πεδίο "kind" και αυξάνει τον αντίστοιχο μετρητή (commit, identity, account, info). Σύμφωνα με την εκφώνηση, μέσα στο νήμα δεν γίνεται καμία εκτύπωση ή εγγραφή σε αρχείο.

Το τρίτο νήμα, ο **Καταγραφέας**, εκτελείται αυστηρά περιοδικά κάθε 1 δευτερόλεπτο με χρήση `clock_nanosleep(CLOCK_REALTIME, TIMER_ABSTIME)`. Σε κάθε αφύπνιση κλειδώνει προσωρινά το mutex των μετρητών, διαβάζει και μηδενίζει τους μετρητές, υπολογίζει την πληρότητα του buffer και τη χρήση CPU, και γράφει μία γραμμή CSV στο `metrics_log.txt` με άμεσο `fflush`.

2.2 Κυκλικός buffer και επιλογή μεγέθους

Ο buffer υλοποιήθηκε ως στατικός πίνακας `ring[512]` [65536] ($512 \text{ θέσεις} \times 64 \text{ KB} = 32 \text{ MB}$) με δείκτες `head/tail` και μετρητή πληρότητας. Επιλέξαμε συνειδητά στατική δέσμευση αντί για `malloc`: ό,τι δεν δεσμεύεται δυναμικά δεν μπορεί να διαρρεύσει, κάτι που είναι κρίσιμο για λειτουργία ημερών. Το μέγεθος προέκυψε από μετρήσεις κατά την ανίχνευση του stream: με μέσο ρυθμό $\sim 40 \text{ Hz}$, οι 512 θέσεις καλύπτουν περίπου 12 δευτερόλεπτα πλήρους στασιμότητας του Καταναλωτή, ενώ τα 64 KB ανά θέση υπερκαλύπτουν το μεγαλύτερο μήνυμα που παρατηρήθηκε. Κατά την εκκίνηση γίνεται `pre-fault` όλου του πίνακα με `memset`, ώστε να μην εμφανίζονται `page faults` κατά τη λειτουργία και το RSS να σταθεροποιείται από την αρχή.

2.3 Αντιμετώπιση race conditions

Χρησιμοποιούμε δύο ξεχωριστά `mutexes`: ένα για τον κυκλικό buffer και ένα για τους τέσσερις μετρητές. Αν υπήρχε ένα κοινό, ο Καταγραφέας που κλειδώνει κάθε δευτερόλεπτο θα καθυστέρουσε τον Παραγωγό που προσπαθεί να αποθηκεύσει πακέτο· με τον διαχωρισμό, τα δύο κρίσιμα τμήματα δεν συγκρούονται ποτέ. Το `condition variable` εξαλείφει το `busy-waiting`, αφού ο Καταναλωτής κοιμάται μέχρι να ειδοποιηθεί. Έτσι κάθε κοινόχρηστη δομή προσπελάζεται αποκλειστικά υπό το αντίστοιχο `mutex` και δεν υφίσταται δυνατότητα ταυτόχρονης εγγραφής.

3 Μετρικές Πραγματικού Χρόνου

Η χρονική σήμανση κάθε γραμμής γίνεται με `clock_gettime(CLOCK_REALTIME)` και αποθηκεύεται σε δύο πεδία (`tv_sec`, `tv_nsec`), όπως ορίζει η εκφώνηση. Η χρήση CPU υπολογίζεται διαβάζοντας την πρώτη γραμμή του `/proc/stat` και παίρνοντας τη διαφορά των `jiffies` (συνολικών και `idle`) μεταξύ δύο διαδοχικών αναγνώσεων, δηλαδή $100 \cdot (\Delta_{total} - \Delta_{idle}) / \Delta_{total}$, οπότε το ποσοστό αφορά ακριβώς το τελευταίο δευτερόλεπτο.

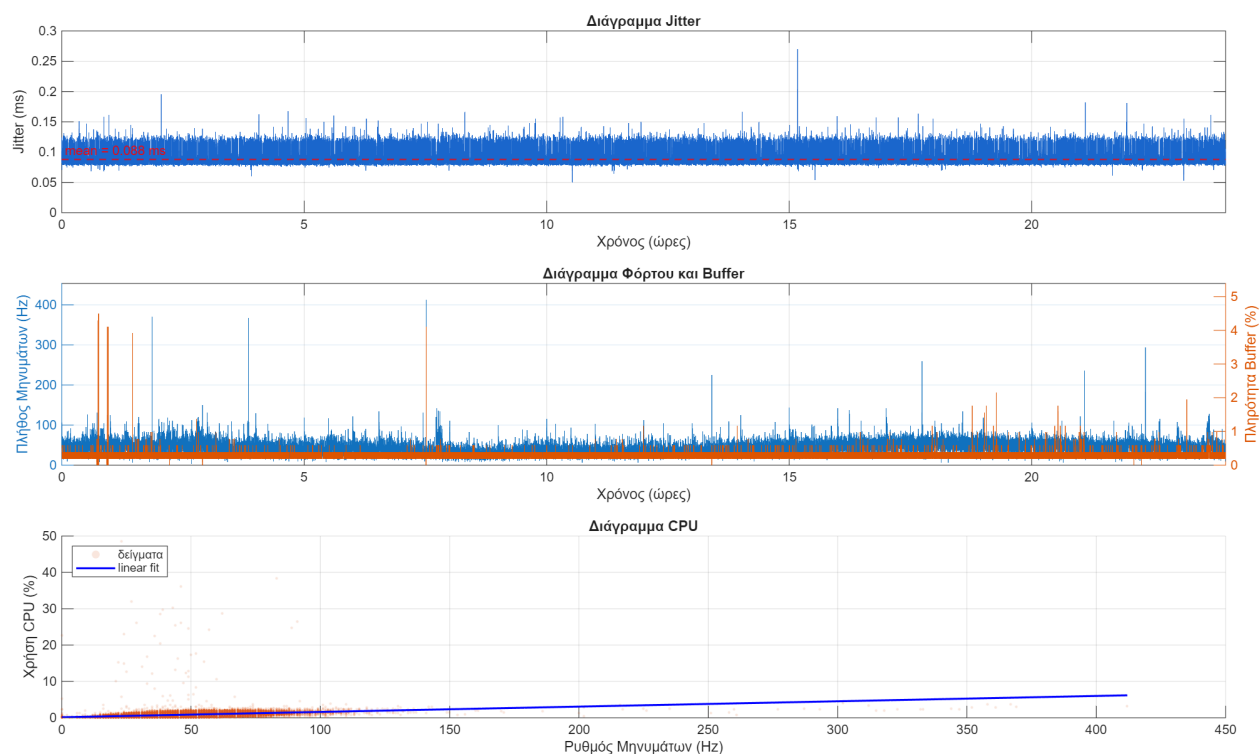
Για την αποφυγή του `drift`, το κλειδί είναι η σημαία `TIMER_ABSTIME`: το νήμα δεν κοιμάται για 1 δευτερόλεπτο αλλά μέχρι μια απόλυτη χρονική στιγμή, η οποία προσαυξάνεται κατά ακριβώς $+1 \text{ s}$ σε κάθε κύκλο. Έτσι, τυχόν καθυστέρηση μιας αφύπνισης δεν μεταφέρεται στην επόμενη και η ολίσθηση είναι δομικά μηδενική. Επειδή μάλιστα το ιδανικό `deadline` έχει πάντα `tv_nsec = 0`, το πεδίο `Nanoseconds` του αρχείου είναι απευθείας το `jitter` σε νανοδευτερόλεπτα, κάτι που απλοποίησε τη μετα-επεξεργασία. Το `jitter` προκύπτει πάντα θετικό, αφού ο πυρήνας δεν ξυπνά ποτέ ένα νήμα νωρίτερα από την προθεσμία του. Τέλος, επειδή το `Pi` δεν διαθέτει `RTC`, αφήσαμε το `NTP` να σταθεροποιηθεί πριν την έναρξη, ώστε κατά τη διάρκεια της καταγραφής να γίνονται μόνο ομαλές διορθώσεις και όχι άλματα.

4 Αποτελέσματα 24ωρης Καταγραφής

Η καταγραφή πραγματοποιήθηκε στις 12/07/2026 (00:00:00–23:59:59) με το πρόγραμμα να τρέχει ως systemd service στο φυσικό hardware του Pi Zero 2W. Το τελικό αρχείο περιέχει ακριβώς 86400 γραμμές, χωρίς κανένα κενό στη χρονοσειρά (επαληθεύτηκε ότι κάθε timestamp διαφέρει ακριβώς κατά 1 από το προηγούμενο), και συνολικά επεξεργάστηκαν 3.735.121 μηνύματα. Ο πίνακας 1 συνοψίζει τα βασικά μεγέθη.

Μέγιστο / μέσο jitter	0.269 ms / 0.088 ms
Drift (χαμένα δευτερόλεπτα)	0
Μέση / μέγιστη χρήση CPU	0.77 % / 48.62 %
Μέγιστη πληρότητα buffer	4.49 %
Μέσος / μέγιστος ρυθμός commit / identity / account / info	43.2 Hz / 412 msg/s

Πίνακας 1: Συγκεντρωτικά αποτελέσματα του 24ώρου.



Σχήμα 1: Διαγράμματα jitter, φόρτου/πληρότητας buffer και συσχέτισης CPU–ρυθμού για το 24ωρο.

Στο πρώτο διάγραμμα παρατηρούμε ότι το jitter παραμένει επίπεδο γύρω από τον μέσο όρο (0.088 ms) για όλο το 24ωρο, με μέγιστη τιμή μόλις 0.269 ms. Η απουσία οποιασδήποτε ανοδικής τάσης επιβεβαιώνει οπτικά και το μηδενικό drift. Στο δεύτερο διάγραμμα, ο ρυθμός των

μηνυμάτων ακολουθεί εμφανή ημερήσιο κύκλο (ηπιότερος τις πρώτες ώρες, εντονότερος από το μεσημέρι και έπειτα) γύρω από μέσο όρο 43.2 Hz, με σποραδικά bursts έως 412 Hz. Οι αιχμές της πληρότητας του buffer συμπίπτουν χρονικά με τις αιχμές του ρυθμού, αναδεικνύοντας την εκρηκτικότητα του δικτύου· ακόμα όμως και στο χειρότερο burst, ο buffer έφτασε μόλις το 4.49 %, δηλαδή έμεινε πάνω από 95 % άδειος. Στο τρίτο διάγραμμα, το scatter δείχνει τη συσχέτιση ρυθμού–CPU: όσο αυξάνονται τα εισερχόμενα μηνύματα, τόσο αυξάνεται η απασχόληση της CPU, με τη γραμμή τάσης να ποσοτικοποιεί το κόστος ανά μήνυμα. Ακόμα και στον μέγιστο ρυθμό, το σύστημα έμεινε πολύ μακριά από κορεσμό.

5 Σταθερότητα και Ανθεκτικότητα Δικτύου

Για τα memory leaks κάναμε δύο ξεχωριστά τεστ πριν την τελική καταγραφή. Στο πρώτο τρέξαμε το πρόγραμμα για 90 δευτερόλεπτα υπό το valgrind, που καταγράφει κάθε δέσμευση και αποδέσμευση μνήμης. Η μικρή διάρκεια αρκεί, γιατί δεν ψάχνουμε πόση μνήμη συσσωρεύεται αλλά αν κάθε δέσμευση έχει την αποδέσμευσή της, και ο Καταναλωτής εκτελεί τον ίδιο ακριβώς κύκλο σε κάθε μήνυμα. Το αποτέλεσμα ήταν 347.135 δεσμεύσεις και ισάριθμες αποδεσμεύσεις, χωρίς ούτε ένα byte αζήτητο και χωρίς κανένα σφάλμα πρόσβασης σε μνήμη, άρα ούτε κίνδυνος για segmentation fault. Στο δεύτερο αφήσαμε το πρόγραμμα να τρέξει κανονικά επί 5 ώρες, μετρώντας τη μνήμη του κάθε 30 δευτερόλεπτα: σταθεροποιείται στα ~44 MB και μετά την πρώτη ώρα η αύξηση πέφτει κάτω από 300 KB/h με φθίνουσα τάση. Βοηθάει ότι ο κυκλικός buffer δεσμεύεται στατικά.

Στο κομμάτι του δικτύου υπάρχει μια παγίδα που δεν είναι προφανής. Ο client μας μόνο ακούει, δεν στέλνει ποτέ τίποτα. Αν λοιπόν κοπεί η σύνδεση, το πρόγραμμα δεν παίρνει κανένα σφάλμα, απλά σταματάει να ακούει και νομίζει ότι όλα είναι εντάξει. Θα κατέγραφε μηδενικά για ώρες χωρίς να το καταλάβει κανείς. Το λύσαμε βάζοντάς τον να ρωτάει περιοδικά «είσαι εκεί;» (PING) και, αν δεν πάρει απάντηση μέσα σε 20 δευτερόλεπτα, να κλείνει τη σύνδεση και να ξαναπροσπαθεί, με ολοένα μεγαλύτερα διαστήματα αναμονής ώστε να μην πιέζει τον server. Το δοκιμάσαμε κόβοντας τεχνητά την κίνηση προς τον Jetstream για 2 λεπτά: η νεκρή σύνδεση εντοπίστηκε στα 20 δευτερόλεπτα και, μόλις επανήλθε το δίκτυο, η ροή ξεκίνησε ξανά μέσα σε 1-2 δευτερόλεπτα. Το πιο σημαντικό όμως είναι ότι ο Καταγραφέας συνέχισε να γράφει κανονικά, μία γραμμή κάθε δευτερόλεπτο, χωρίς να χάσει τίποτα. Οι ίδιες αποσυνδέσεις συνέβησαν αυθόρμητα και μέσα στο κανονικό 24ωρο και αντιμετωπίστηκαν με τον ίδιο τρόπο.

6 Παραδοτέα

Στα παραδοτέα αρχεία συμπεριλαμβάνονται ο πλήρης πηγαίος κώδικας `firehose.c` με το αντίστοιχο `Makefile`, το αρχείο δεδομένων `metrics_log.txt` των 86400 γραμμών, καθώς και ο κώδικας MATLAB που παράγει τα διαγράμματα που παρουσιάστηκαν. Τα παραπάνω είναι επίσης διαθέσιμα στο GitHub: <https://github.com/mpillis/Real-Time-Embedded-Systems>